

# **Python**

# **Data Types**

**Interview Questions & Answers**

**30 Interview Questions with Answers**

*Conceptual • Interview-Level • Output-Based*

*A focused set for interview preparation and conceptual understanding of Python data types.*

### **1. What are the different categories of data types available in Python?**

Answer: Python's built-in types include numeric types (int, float, complex), bool, str, sequence types, set types, mapping types, binary types, and the special NoneType. Python also supports user-defined types, such as classes.

### **2. What is the difference between built-in data types and user-defined data types in Python?**

Answer: Built-in types are provided by Python, such as int, float, str, bool, and NoneType. User-defined types are created by the programmer, commonly with a class. For example, class Student creates a new user-defined type.

### **3. Is Python a dynamically typed or statically typed language? Explain with an example.**

Answer: Python is dynamically typed. A variable does not need a type declaration, and the same name can later refer to an object of another type. For example, x=10 followed by x='hello' is valid.

### **4. What happens to the data type of a variable when we assign a new type of value to it?**

Answer: The name starts referring to the new object, so its current type becomes the type of that object. Python variables are names bound to objects; the name itself does not have a permanently fixed type.

### **5. How does Python determine the data type of a variable at runtime?**

Answer: Python objects carry type information at runtime. When a value is created, it is an object of a particular type, and a variable name refers to that object. The type() function can inspect the object's type.

### **6. What is the difference between type and value in Python?**

Answer: The value is the actual data held by an object, while its type describes what kind of object it is and what operations are supported. In x=10, 10 is the value and its type is int.

### **7. What is the purpose of the type() function? Can it be used to check whether two variables have the same data type?**

Answer: type() returns the type of an object. Yes, types can be compared using type(a)==type(b). For checking whether an object belongs to a type or its subclasses, isinstance() is generally more flexible.

### **8. What is the difference between type() and isinstance() in Python?**

Answer: type(x) returns the exact type of x. isinstance(x,T) checks whether x is an instance of T or a subclass of T, and it can accept a tuple of types. Thus isinstance() is usually preferred for type-hierarchy checks.

### **9. What is type conversion in Python? Why is it required?**

Answer: Type conversion means converting a value from one data type to another when the conversion is supported. It is useful when data is received in a different type than required, such as converting user input from str to int before arithmetic.

### **10. What is the difference between implicit type conversion and explicit type conversion?**

Answer: Implicit conversion is performed automatically in compatible operations, such as promoting an integer to a float in 5+2.5. Explicit conversion is requested by the programmer using functions such as int(), float(), or str().

### **11. Which data types can be converted from one type to another using functions such as int(), float(), str() and bool()?**

Answer: Many common values can be converted when the value is valid for the target type. For example, int('10'), float('10.5'), str(10), and bool(1) are valid. Invalid conversions can raise exceptions such as ValueError or TypeError.

### **12. What happens when we execute x=int('10.5')? Why does Python behave this way?**

```
x=int("10.5")
```

Answer: It raises ValueError. The int() constructor accepts a string representing an integer, such as '10', but '10.5' represents a decimal number and is not a valid integer literal for direct int conversion.

### **13. What will be the data type of z in x=10, y=2.5, z=x+y? Why?**

```
x=10  
y=2.5  
z=x+y
```

Answer: z is a float. Python promotes the integer operand to a compatible numeric type for the arithmetic operation, so the result is 12.5, a floating-point value.

### **14. Why does division using / generally produce a float in Python, even when both operands are integers?**

Answer: In Python 3, / performs true division and returns a floating-point result for ordinary numeric operands. Therefore 10/2 produces 5.0 rather than 5.

### **15. What is the difference between / and // with respect to their result data types?**

Answer: / performs true division and returns a float for integer operands. // performs floor division; with integer operands its result is an int, while with floating-point operands it is a float. Floor division rounds toward negative infinity.

### **16. What is the bool data type in Python? How does Python decide whether a value is True or False?**

Answer: bool represents True and False. When Python needs a truth value, it follows an object's truth-value rules. Zero numeric values, None, False, and empty strings/collections are common examples of falsy values.

### **17. Which types of values are considered Falsy in Python?**

Answer: Common falsy values include False, None, numeric zero values such as 0, 0.0 and 0j, and empty strings and empty built-in collections such as "", [], (), {}, and set(). Other objects are generally truthy unless their type defines otherwise.

### **18. Is bool an independent data type in Python, or is it related to another built-in type? Explain.**

Answer: bool is a distinct built-in type, but it is a subclass of int. Therefore isinstance(True,int) is True, and in numeric contexts True behaves like 1 while False behaves like 0.

### **19. What is the NoneType data type? When is None commonly used?**

Answer: None is the single value of the NoneType type. It commonly represents the absence of a value, such as a function that does not explicitly return a value or a value that is not yet available.

### **20. What is the difference between None and 0 in Python?**

Answer: None means absence of a value and has type `NoneType`. 0 is an integer and has type `int`. Both are falsy, but they represent different concepts and `None==0` is `False`.

## 21. What is the difference between None and False in Python?

Answer: None has type `NoneType` and represents absence of a value, while False has type `bool` and represents a false Boolean value. Both are falsy, but `None==False` is `False`.

## 22. What is the complex data type in Python? How are its real and imaginary parts represented?

Answer: complex represents a number with real and imaginary parts. Python uses `j` for the imaginary part, for example `3+4j`. The `.real` and `.imag` attributes access the two parts.

## 23. What will be the output and data type of x=5/2 and y=5//2?

```
x=5/2
y=5//2
print(x, type(x))
print(y, type(y))
```

Answer: x is 2.5 and its type is `float`. y is 2 and its type is `int` because both operands are integers.

## 24. What happens when we perform an operation between incompatible data types such as 10+'20'? Why?

```
10+"20"
```

Answer: It raises `TypeError`. Python does not define addition between an integer and a string because they represent different kinds of data. Explicit conversion is required if that is what the program intends.

## 25. Why does Python not automatically convert a string containing a number into an integer during an arithmetic operation?

Answer: A string is text, even when its characters represent a number. Automatically guessing that numeric-looking strings should become numbers could change program meaning unexpectedly, so Python requires explicit conversion such as `int('20')`.

## 26. What happens when we try to convert a floating-point value to an integer using int()? Does Python round the value or truncate it?

Answer: `int()` truncates the fractional part; it does not round to the nearest integer. For example, `int(10.9)` is 10 and `int(-10.9)` is -10. The conversion removes the fractional portion toward zero.

## 27. Can every Python data type be converted into every other data type? What determines whether a type conversion is valid?

Answer: No. A conversion is valid only when the target type supports a meaningful conversion for the given value. For example, `int('25')` works, while `int('hello')` raises `ValueError`. Some unsupported conversions can raise `TypeError`.

## 28. What is the difference between == and is when comparing values of different or same data types?

Answer: `==` compares values for equality, while `is` checks whether two names refer to the same object. For example, `1==True` is `True`, but `1 is True` is `False`. `is` is commonly used for identity checks such as `x is None`.

## 29. Why can two objects have the same value and same data type but still be different objects?

Answer: Two separate objects can contain equal data while having different object identities. Therefore `a==b` can be `True` while `a is b` is `False`. Python may reuse some objects as an optimization, so identity should not be inferred from equality.

**30. What is the difference between dynamic typing and strong typing in Python? Why is Python considered dynamically typed but strongly typed?**

Answer: Dynamic typing means types are associated with runtime objects and variable names do not require fixed type declarations. Strong typing means Python does not silently perform arbitrary incompatible conversions, such as treating '10' as the integer 10 in '10'+5. Thus Python is dynamically typed and strongly typed.